

EXAM ANSWER SHEET

MATRICULATION :

FULL NAME :

GROUP:

EXERCISE 1 (7.5 POINTS).

Main Program	<pre> PROGRAM pgm1 CONST Tmax = 100 VAR x:integer; Ta: array [Tmax] of integer; Tb: array [Tmax] of integer; a, b: integer BEGIN a ← 360 ; b ← 9702; InitArray(&Ta, Tmax); Decomp_prime(&Ta, Tmax, a) InitArray(&Tb, Tmax); Decomp_prime(&Tb, Tmax, b) print ("GCD (a, b) = ", GCD_decomp(Ta, Tb, Tmax)) print("LCM (a, b) = ", LCM_decomp(Ta, Tb, Tmax)) END </pre>	
Prime module	FUNCTION Prime (x: integer) : Boolean (1.5 points)	
	<pre> VAR i, cpt: integer BEGIN cpt ← 0; for i ← 2 to (x↑(0.5)) do if (x mod i = 0) then cpt ++; return(cpt = 0); END </pre>	<pre> VAR i, cpt: integer BEGIN cpt ← 0; for i ← 2 to (x div 2) do if (x mod i = 0) then return false; return(true); END </pre>
InitArray module (1.5 points)	<pre> PROCEDURE InitArray (VAR T: array [] of integer, n: integer) VAR x, cpt: integer BEGIN </pre>	
	<pre> cpt ← 0; x ← 2; while (cpt<n/2) do Begin if (Prime(x)) then begin T[2*cpt] ← x; T[2*cpt + 1] ← 0; cpt++; end x++; end END </pre>	<pre> For cpt←0 to n - 1 do If (cpt mod 2 = 0) then begin T[cpt] ← x; do x++ while (prime (x)) end else T[cpt] ← 0; </pre>
Decomp prime	<pre> PROCEDURE Decomp_prime (VAR T: array [] of integer, n, x: integer) VAR puis, i : integer BEGIN </pre>	

	<pre> i ← 0; while ((x>1)AND(i<n))do begin puis ← 0; while (x mod T[i]=0) do begin puis++; x←x div T[i]; end T[i+1]←puis; i←i+2; end END </pre>	<pre> For i←0 to n-1 step 2 do While (x mod T[i] = 0) do Begin T[i+1]++; x←x div T[i] end </pre>
GCD_decomp module (1.5 points)	<pre> FUNCTION GCD_decomp (Ta: array[] of integer, Tb: array [] of integer, n: integer) : integer VAR i, m, d: integer BEGIN d ← 1; </pre>	
	<pre> for i←0 to (n/2 – 1) do begin m ← Ta[2*i+1]; if (m>Tb[2*i+1]) then m ← Tb[2*i+1]; d ← d * (Ta[2*i]↑ m); end return d; END </pre>	<pre> for i←1 to n – 1 step 2 do begin m ← Ta[i]; if (m>Tb[i]) then m ← Tb[i]; d ← d * (Ta[i-1]↑ m); end return d; END </pre>
LCM_decomp module (1.5 points)	<pre> FUNCTION LCM_decomp (Ta: array[] of integer, Tb: array [] of integer, n: integer) : integer VAR i, m, r : integer BEGIN r ← 1; </pre>	
	<pre> for i←0 to n/2 – 1 do begin m ← Ta[2*i+1]; if (m<Tb[2*i+1]) then m ← Tb[2*i+1]; r ← r * (Ta[2*i]↑ m); end return r; END </pre>	<pre> for i←1 to n – 1 step 2 do begin m ← Ta[i]; if (m<Tb[i]) then m ← Tb[i]; r ← r * (Ta[i-1]↑ m); end return r; END </pre>

EXERCISE 2 (6 POINTS).

You can use (without rewriting) the following modules¹ :

- **bool Prime(int x)** which checks if a number x is prime or not.

¹ Vous pouvez utiliser (sans les réécrire) les modules suivants :

- **bool Prime(int x)** qui permet de vérifier si un nombre x est premier ou non.
- **void SwapArray(int T[], int i, int j)** qui permet de permuter les valeurs de deux cases (i, j).

- **void SwapArray(int T[], int i, int j)** which swaps the values of two cells (i, j).

Main Program (code 2)	<pre>int main() { int T [] = {2313, 8345, 1020, 6131, 4532, 8763, 2348}; int n = sizeof(T) / sizeof(T[0]); RADIX(T, n, 4); Interclassify(T,n); WriteArray(T, n); return 0; }</pre>
Biprime module (1.5 points)	<pre>bool Biprime (int N) { int N1, N2; N1 = N /100; N2 = N % 100; if ((N1>=10)&&(N2>=10)&&(Prime(N1))&&(Prime(N2))) return true; else return (false); }</pre>
Interclassify module (2.5 points)	<pre>void Interclassify(int T[], int n) { int i, p, d , f; d = 0; for (i=0; i<n; i++) if (Biprime(T[i])) { SwapArray(T, d, i); d++; } f = n-1; for (i=d; i<=f; i++) if (T[i]%2==0){ SwapArray(T, f, i); f--; } }</pre>

EXERCISE 3 (4.5 POINTS).

Main Program (code 3)	<pre>const int Tmax = 4; int main(){ int M[Tmax][Tmax]; ReadMATRIX (M, Tmax); WriteMATRIX (M, Tmax); if (Diagonal(M, Tmax)) printf("This matrix is diagonal \n"); if (Symmetric(M, Tmax)) printf("This matrix is symetric \n"); if (Diagonal(M, Tmax)) printf("This matric is magic \n"); return 0; }</pre>	
Existing Modules	<pre>void ReadMATRIX (int M[Tmax][Tmax], int n) { srand(time(NULL)); for (int row = 0; row < n; row++) for (int col = 0; col < n; col++) M[row][col] = rand() % 100;}</pre>	<pre>void WriteMATRIX (int M[Tmax][Tmax], int n) { for (int row = 0; row < n; row++) for (int col = 0; col < n; col++) printf("%d ", M[row][col]); }</pre>

Diagonal module (1 points)	<pre> bool Diagonal (int M[Tmax][Tmax], int n) { int col, row = 0; bool prop = true; while ((row < Tmax) &&(prop)) { col = 0; while((col < Tmax) &&(prop)) { if (((row != col)&&(M[row][col]!=0)) ((row == col)&&(M[row][col]==0))) prop = false; col++; } row++; } return prop;} </pre>
Symmetric module (1 points)	<pre> bool Symmetric (int M[Tmax][Tmax], int n) { int col, row = 0; bool prop = true; while ((row < Tmax) &&(prop)) { col = 0; while((col < Tmax) &&(prop)) { if (M[row][col]!=M[col][row]) prop = false; col++; } row++;} return prop;} </pre>
Magic module (2.5 points)	<pre> bool Magic (int M[Tmax][Tmax], int n) { int i, j, SR, SC, mag_const = 0; bool prop = true; //calculate the magic constant (mag_const) for (i=0; i<n; i++) mag_const+=M[i][i]; //calculate the sum of rows (SR), the sum of columns (SC), and compare them with the magic constant while ((i < n) &&(prop)) { SR = 0; SC = 0; for (j = 0; j<n; j++) { SR+= M[i][j]; SC+= M[j][i];} if ((SR != mag_const) (SC != mag_const)) prop = false; i++;} return prop;} </pre>

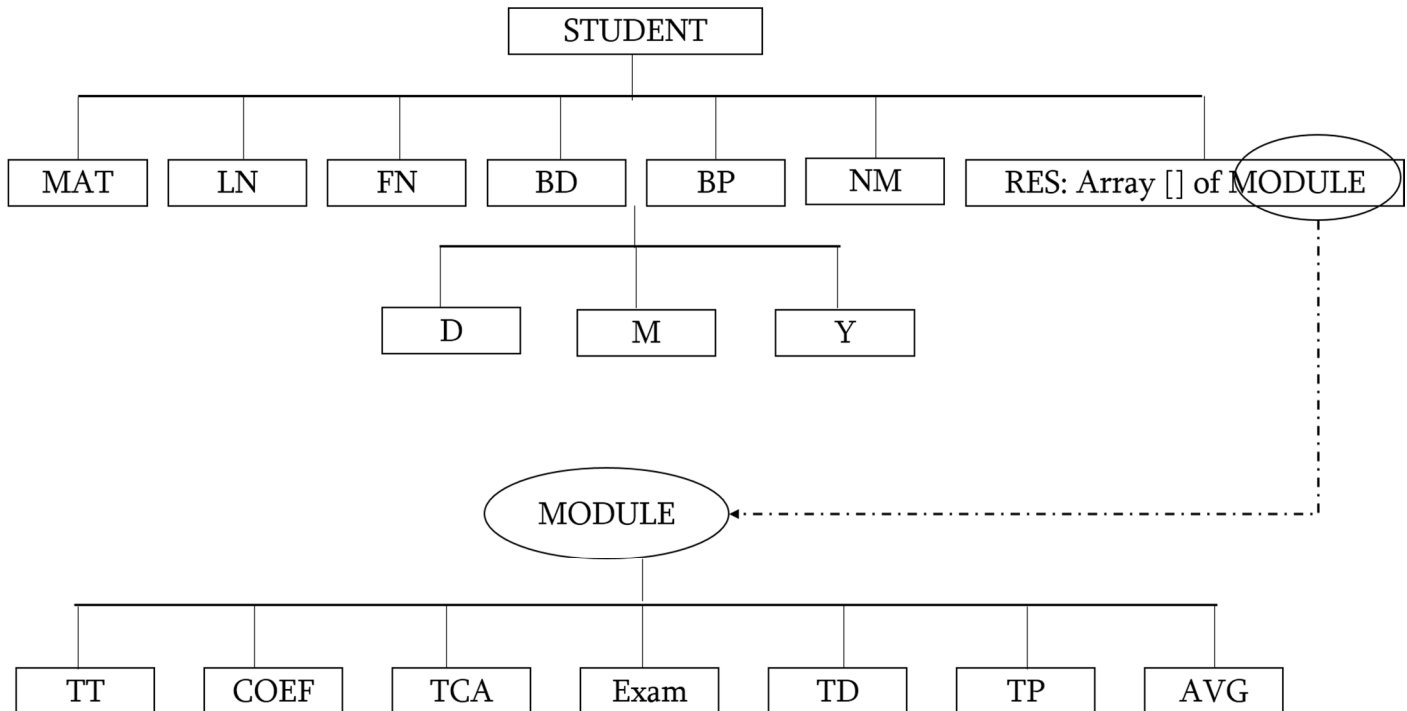
EXAM ANSWER SHEET

MATRICULATION :

FULL NAME :

GROUP:

EXERCISE 4 (5 POINTS).



- **STUD**: Student (*Etudiant*)
- **LN**: Last Name (*Nom*)
- **BD**: Birth Date [D: Day, M: Month, Y: Year] (*Date de naissance [Jour, Mois, Année]*)
- **BP**: Birth Place (*Lieu de naissance*)
- **NM**: Number of Module (*Nombre des modules*)
- **TT**: Title (*Intitulé*)
- **TCA**: Type of Continuous Assessment (*Type de contrôle continu*)
- **EXAM** (*Examen*)
- **TD**: Directed work (*Travaux Dirigés*)
- **MAT**: Matriculation (*Matricule*)
- **FN**: First Name (*Prénom*)
- **RES**: Results (*Résultats*)
- **COEF**: COEFFICIENT
- **AVG**: Average (*moyenne*)
- **TP**: Practical work (*Travaux Pratiques*)

1. Create a data structure to store the student's information. (2.5 points)

Type student = structure (1.25 points)

Begin

MAT, NM : integer
LN, FN, BP : array [100] of char
BD: Date
RES: array [10] of MODULE

End

Type Date = structure (0.5 points)

Begin

D: 1 .. 31

2. Write the function "avg_module" to calculate the average of a module (1.25 points)

Function avg_semestre (m:Module): real

Var moy: real

Begin

Switch (m.TCA)

Begin

M: (Jan, Fev, Mars,, Dec)

A: integer

End

Type Module = structure (1.75 points)

Begin

TT: array [20] of char

Coeff, TCA: integer

Exam, TD, TP, AVG: real

End

```

Case 0 : moy ← m.Exam
Case 1 : moy ← 0,6 * m.Exam + 0,4 * m.TD
Case 2 : moy ← 0,6 * m.Exam + 0,4 * m.TP
Case 3 : moy ← 0,6 * m.Exam + 0,4 * (m.TD+m.TP)/2
Default moy ← 0

```

```

End
Return (moy)

```

```

End

```

3. Write the function "avg_semestre " to calculate the semester average. (1.25 points)

```

Function avg_semestre (ms :array[] of Module, NM :integer): real

```

```

Var

```

```

Begin

```

```

    som ← 0; coef ← 0

```

```

    For i ← 0 to NM - 1 do

```

```

        Begin

```

```

            som ← som + (ms.module[i].AVG * ms.module[i].coef)

```

```

            coef ← coef + ms.module[i].coef

```

```

        End

```

```

    Return (som/coef)

```

```

End

```